

Projet LLMOps — Déploiement et surveillance d'un assistant conversationnel

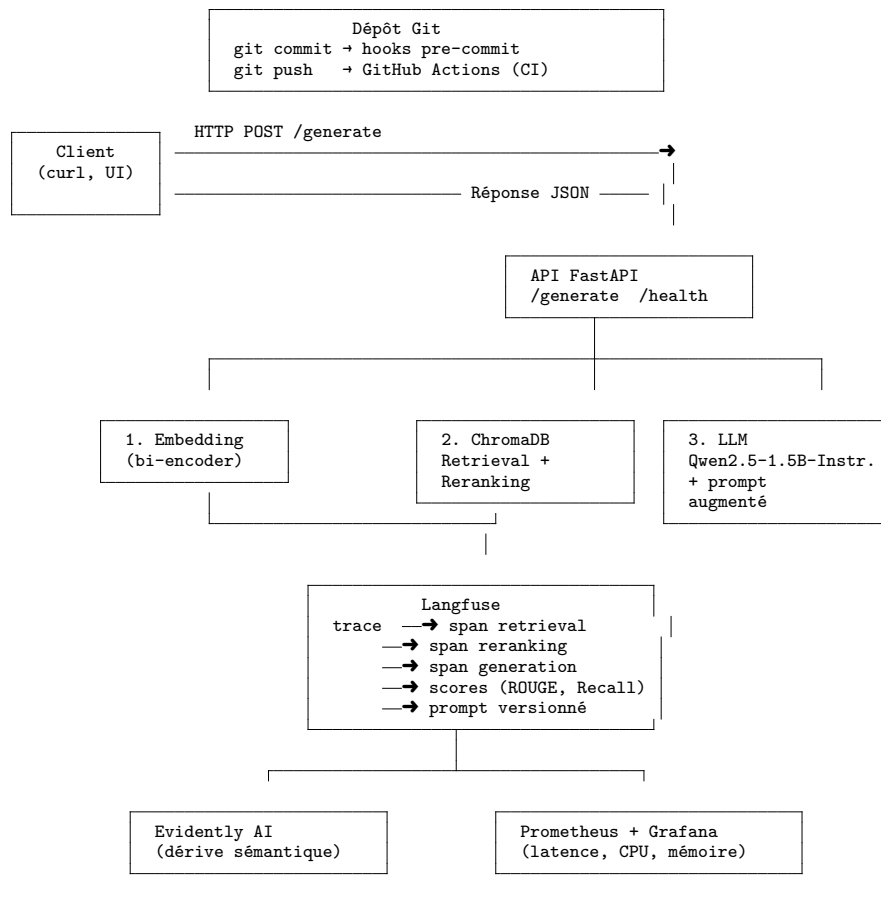
Contexte

Une organisation souhaite déployer un assistant conversationnel pour son service client. Pour améliorer la pertinence des réponses, l'assistant doit être capable de **retrouver dynamiquement les informations pertinentes** dans une base de connaissance (FAQ, politiques, fiches produit) avant de générer sa réponse. C'est le principe du **RAG** (*Retrieval-Augmented Generation*).

Le pipeline complet couvre six dimensions du LLMOps :

Dimension	Outil principal	Objectif
Base de connaissance	ChromaDB + sentence-transformers	Recherche sémantique locale
Qualité automatisée	pre-commit	Bloquer les erreurs avant le commit
Intégration continue	GitHub Actions	Exécuter les tests à chaque push
Déploiement	FastAPI	Exposer le pipeline RAG via une API REST
Observabilité LLM	Langfuse	Tracer chaque appel RAG, gérer les prompts, suivre les scores
Surveillance	Evidently AI / Prometheus + Grafana	Détecter les dérives, superviser le système

Architecture cible



Structure du dépôt attendue

```

mon_projet_llmops/
├── .github/
│   └── workflows/
│       ├── tests.yaml           # CI : tests unitaires + couverture
│       ├── codecheck.yaml       # CI : pre-commit (qualité de code)
│       └── rag_eval.yaml        # CI : évaluation RAG + log Langfuse
├── .pre-commit-config.yaml
├── .env                         # Clés Langfuse (jamais committé !)
├── pyproject.toml              # Config ruff, mypy, pytest
├── src/
│   ├── __init__.py
│   ├── tokeniseur.py
│   ├── modele.py
│   └── prompts.py
├── rag/
│   ├── __init__.py
│   ├── base_connaissance.py     # Création et alimentation de ChromaDB
│   ├── chunking.py             # Découpage des documents en passages
│   ├── recherche.py            # Recherche sémantique (bi-encoder)
│   ├── reranking.py            # Reclassement fin (cross-encoder)
│   └── pipeline.py             # Assemblage RAG complet (avec traces Langfuse)
├── evaluation/
│   ├── jeu_evaluation.jsonl     # Questions + réponses de référence
│   ├── evaluer_rag.py          # Métriques + envoi scores vers Langfuse
│   └── rapport_eval.json       # Résultats de l'évaluation
├── api/
│   └── app.py                  # API instrumentée avec Langfuse
├── monitoring/
│   ├── analyse_derive.py       # Drift Evidently (données exportées de Langfuse)
│   ├── prometheus_app.py
│   └── docker-compose.yml      # Prometheus + Grafana + Langfuse (optionnel)
├── tests/
│   ├── __init__.py
│   ├── dataset.py
│   ├── test_tokeniseur.py
│   ├── test_modele.py
│   ├── test_rag.py
│   └── test_chunking.py
├── data/
│   ├── faq_service_client.jsonl
│   ├── documents_longes.jsonl
│   └── corpus_reference.jsonl
├── requirements.txt
├── requirements_tests.txt
└── README.md

```

Partie 0 — RAG : base de connaissance vectorielle locale

Concepts

Qu'est-ce que le RAG ? Un LLM génère du texte à partir de ce qu'il a appris pendant son entraînement. Mais ses connaissances sont figées à une date donnée, et il ne connaît pas les données spécifiques à votre organisation (FAQ interne, politiques de remboursement, catalogue produit...).

Le **RAG** (*Retrieval-Augmented Generation*) résout ce problème en deux étapes :

1. **Recherche** (*Retrieval*) : à partir du prompt de l'utilisateur, on recherche dans une base de documents les passages les plus pertinents.
2. **Génération augmentée** (*Augmented Generation*) : on injecte ces passages dans le prompt avant de l'envoyer au LLM, qui peut ainsi répondre avec des informations factuelles et à jour.

Sans RAG :
Prompt : "Quel est le délai de retour ?"
LLM → réponse inventée ou vague

Avec RAG :
Prompt : "Quel est le délai de retour ?"
→ Recherche dans la FAQ → "Retours acceptés sous 30 jours [doc FAQ-07]"
→ Prompt augmenté : "Contexte : [doc FAQ-07]. Question : Quel est le délai de retour ?"
LLM → réponse ancrée dans un fait réel

Comment fonctionne la recherche sémantique ? La recherche sémantique ne cherche pas les mots exacts (comme **grep**), mais le **sens** de la phrase. Elle repose sur les *embeddings* : des représentations numériques (vecteurs) d'une phrase dans un espace de haute dimension, où les phrases sémantiquement proches sont proches géométriquement.

```
"Je veux renvoyer un article"      → [0.12, -0.45, 0.87, ...]
"Comment retourner un produit ?"   → [0.14, -0.43, 0.85, ...] ← proche !
"Recette de quiche lorraine"        → [-0.72, 0.31, -0.12, ...] ← loin
```

Le modèle d'embedding recommandé est **all-MiniLM-L6-v2** de la bibliothèque **sentence-transformers** : ~80 Mo, très rapide sur CPU.

ChromaDB **ChromaDB** est une base de données vectorielle open-source, entièrement locale. Elle permet de :

- stocker des documents avec leurs embeddings ;
- retrouver les **top-k** documents les plus proches d'un vecteur de requête ;
- filtrer par métadonnées.

ChromaDB fonctionne en mode persistant (données sauvegardées sur disque) ou en mémoire.

```
Base ChromaDB
├── Collection "faq_service_client"
│   ├── doc_1 : "Les retours sont acceptés sous 30 jours..." + embedding
│   ├── doc_2 : "La livraison standard prend 3 à 5 jours..." + embedding
│   ├── doc_3 : "Pour annuler une commande, contactez..."   + embedding
│   └── ...
```

Exercice 0.1 — Créer la base de connaissance

Installer les dépendances RAG :

```
pip install chromadb sentence-transformers
```

Votre objectif : créer un fichier `rag/base_connaissance.py` qui : 1. Charge le fichier `data/faq_service_client.jsonl` (voir format ci-dessous) 2. Initialise un client ChromaDB en mode persistant (les données sont sauvegardées dans un dossier `data/chroma_db/`) 3. Crée (ou ouvre si elle existe déjà) une *collection* ChromaDB 4. Calcule les embeddings des documents avec `sentence-transformers` 5. Insère les documents dans la collection

Format de la FAQ (`data/faq_service_client.jsonl`) :

```
{
  "id": "faq-01", "question": "Quel est le délai de retour ?", "reponse": "Vous pouvez",
  ↪ "retourner tout article dans un délai de 30 jours suivant la réception de votre",
  ↪ "commande.", "categorie": "retours"}
{"id": "faq-02", "question": "Comment suivre ma commande ?", "reponse": "Un email de",
  ↪ "confirmation avec un lien de suivi vous est envoyé dès l'expédition.", "categorie":",
  ↪ "livraison"}
{"id": "faq-03", "question": "Puis-je modifier mon adresse de livraison ?", "reponse":",
  ↪ "L'adresse peut être modifiée uniquement si la commande n'a pas encore été expédiée.",",
  ↪ "categorie": "livraison"}
{"id": "faq-04", "question": "Comment obtenir un remboursement ?", "reponse": "Le",
  ↪ "remboursement est effectué sous 5 à 10 jours ouvrés après réception de l'article",
  ↪ "retourné.", "categorie": "remboursement"}
{"id": "faq-05", "question": "Livrez-vous à l'international ?", "reponse": "Nous livrons",
  ↪ "dans 30 pays. Les délais varient entre 7 et 15 jours ouvrés selon la destination.",",
  ↪ "categorie": "livraison"}
{"id": "faq-06", "question": "Que faire si mon colis est endommagé ?", "reponse": "Prenez",
  ↪ "des photos du colis et contactez notre support dans les 48h. Nous organisons un renvoi",
  ↪ "gratuit.", "categorie": "litiges"}
{"id": "faq-07", "question": "Comment annuler une commande ?", "reponse": "Une commande peut",
  ↪ "être annulée dans les 2 heures suivant sa validation. Passé ce délai, contactez le",
  ↪ "support.", "categorie": "commandes"}
{"id": "faq-08", "question": "Proposez-vous des cartes cadeau ?", "reponse": "Oui, des",
  ↪ "cartes cadeau de 10 à 200 € sont disponibles sur notre site.", "categorie": "produits"}
```

Indices :

- Cherchez dans la documentation ChromaDB comment instancier un `PersistentClient` avec un chemin de dossier.
- La méthode permettant de créer ou récupérer une collection s'appelle `get_or_create_collection`. Réfléchissez à la fonction de distance à utiliser pour des embeddings normalisés (cosine vs l2).
- `SentenceTransformer("all-MiniLM-L6-v2").encode(liste_de_textes)` retourne un tableau NumPy. ChromaDB attend une liste de listes.
- Chaque document dans ChromaDB a besoin d'un identifiant unique (`ids`), d'un texte (`documents`) et optionnellement de métadonnées (`metadatas`).
- Avant d'insérer, vérifiez si la collection est déjà peuplée pour éviter les doublons.

Question de réflexion : pourquoi faut-il encoder la *question* du FAQ plutôt que la *réponse* ? Qu'est-ce que cela implique sur la similarité calculée à l'étape de recherche ?

Exercice 0.2 — Recherche sémantique

Votre objectif : créer `rag/recherche.py` avec une fonction `_rechercher_documents(requete, collection, modele_embedding, top_k)` qui : 1. Encode la requête utilisateur en vecteur 2. Interroge la collection ChromaDB pour obtenir les `top_k` documents les plus proches 3. Retourne les textes des documents trouvés et leur score de similarité

Indices :

- ChromaDB expose une méthode `query()` sur la collection. Elle accepte `query_embeddings` ou `query_texts`. Quelle est la différence ?
- Le résultat de `query()` contient une clé `distances` : une distance faible signifie une forte similarité (en distance cosinus normalisée). Comprenez bien l'interprétation selon la métrique choisie.
- Pensez à filtrer les résultats dont le score de similarité est en dessous d'un seuil minimum, pour éviter d'injecter des documents non pertinents.
- La signature de la fonction doit permettre de choisir `top_k` dynamiquement (valeur par défaut recommandée : 3).

Question de réflexion : que se passe-t-il si la requête de l'utilisateur est dans un domaine complètement différent de la base (ex. : une question sur la météo) ? Comment le pipeline doit-il se comporter ?

Exercice 0.3 — Assemblage du pipeline RAG

Votre objectif : créer `rag/pipeline.py` avec une fonction `_generer_avec_rag(prompt_utilisateur, collection, modele_embedding, modele_llm, tokeniseur)` qui : 1. Recherche les documents pertinents (appel à `rag/recherche.py`) 2. Construit un *prompt augmenté* qui injecte le contexte retrouvé 3. Génère la réponse avec le LLM 4. Retourne la réponse et les documents sources utilisés

Indices sur la construction du prompt augmenté :

Le prompt augmenté doit suivre une structure claire pour que le LLM comprenne ce qui est contexte et ce qui est la question. Une structure couramment utilisée est :

Tu es un assistant service client. Réponds uniquement en te basant sur le contexte fourni. Si la réponse n'est pas dans le contexte, réponds : "Je n'ai pas l'information."

Contexte :

```
[1] <texte du document 1>
[2] <texte du document 2>
[3] <texte du document 3>
```

Question : <prompt de l'utilisateur>

Réponse :

Indices :

- La construction du prompt est du simple formatage de chaînes Python (pas de bibliothèque spéciale nécessaire).
- Le LLM doit générer à partir du prompt augmenté, pas du prompt original. Où dans le code faut-il substituer l'un à l'autre ?
- Gardez une trace des IDs des documents sources utilisés pour permettre la traçabilité (quels passages ont été injectés ?).
- Réfléchissez au cas dégénéré : que faire si ChromaDB ne retourne aucun document au-dessus du seuil de pertinence ?

Exercice 0.4 — Tests unitaires du pipeline RAG

Votre objectif : créer `tests/test_rag.py` avec les tests suivants :

Test	Ce qu'il vérifie
<code>test_recherche_retourne_resultats</code>	La recherche retourne bien <code>top_k</code> résultats pour une requête pertinente
<code>test_recherche_pertinence</code>	Pour “retourner un article”, le document FAQ-01 est dans les résultats
<code>test_recherche_hors_domaine</code>	Pour une requête hors domaine, aucun document ne dépasse le seuil
<code>test_prompt_augmente_contient_contenu</code>	Le prompt augmenté contient bien les textes des documents trouvés
<code>test_pipeline_complet_retourne_str</code>	Le pipeline complet retourne une chaîne de caractères non vide

Indices :

- Pour isoler les tests, utilisez une *collection de test* distincte de la collection de production. Comment créer une collection temporaire en mémoire dans ChromaDB ?
 - Pour `test_recherche_pertinence`, la vérification ne doit pas être stricte sur le rang exact — vérifiez seulement que le document *apparaît* dans les résultats.
 - Pour `test_recherche_hors_domaine`, définissez une requête clairement hors périmètre et vérifiez que tous les scores de similarité sont en dessous du seuil.
 - `pytest.fixture` est adapté pour initialiser une collection ChromaDB en mémoire peuplée de quelques documents de test.
-

Exercice 0.5 — Chunking : découpage des documents longs

Concept La FAQ de service client est composée de courtes entrées. Mais dans un contexte réel, les documents sources (politiques internes, manuels produits, contrats de service) peuvent faire plusieurs pages. Indexer un document entier dans ChromaDB crée deux problèmes :

1. **Fenêtre de contexte saturée** : le LLM ne peut traiter qu'un nombre limité de tokens. Injecter un document de 10 pages dépasse sa capacité.
2. **Bruit sémantique** : un grand document traite de plusieurs sujets. Son embedding est une moyenne de tous ces sujets et devient moins précis pour chacun.

Le **chunking** consiste à découper un document en *passages* (*chunks*) plus petits avant l'indexation, chaque chunk ayant son propre embedding.

Document original (2000 mots)

Paragraphe 1 (200 mots)	
Paragraphe 2 (200 mots)	→ Chunk 1 [embedding_1]
Paragraphe 3 (200 mots)	→ Chunk 2 [embedding_2]
...	→ Chunk 3 [embedding_3]
Paragraphe 10 (200 mots)	→ ...

Recherche : "délai de retour"

→ Chunk 3 correspond (score 0.91)
plutôt que l'embedding bruité du document entier

Paramètres clés du chunking :

Paramètre	Description	Valeur typique
<code>_taille_chunk</code>	Nombre de tokens ou de mots par chunk	200–500 tokens
<code>_chevauchement</code>	Nombre de tokens partagés entre deux chunks consécutifs	10–20 % de <code>_taille_chunk</code>

Le **chevauchement** (*overlap*) évite de couper une phrase à la frontière entre deux chunks et de perdre le contexte de transition.

Sans chevauchement :

```
Chunk 1 : [----- texte A -----]
Chunk 2 : [----- texte B -----]
```

Avec chevauchement de 50 mots :

```
Chunk 1 : [----- texte A -----]
Chunk 2 : [-- fin A -- texte B -----]
```

50 mots partagés

Votre objectif : créer `rag/chunking.py` avec une fonction `_decouper_document(texte, taille_chunk, chevauchement)` qui : 1. Découpe le texte en une liste de chunks de taille `taille_chunk` (en mots) 2. Applique un chevauchement de

chevauchement mots entre les chunks consécutifs 3. Retourne la liste des chunks sous forme de chaînes de caractères

Puis créer une fonction `_indexer_documents_longes(chemin_jsonl, collection, modele_embedding, taille_chunk, chevauchement)` qui :

1. Charge un fichier JSONL contenant des documents longs
2. Découpe chaque document en chunks
3. Insère chaque chunk individuellement dans ChromaDB avec comme métadonnée l'ID du document parent

Indices :

- Commencez par tokeniser le texte en mots (`.split()` suffit comme approximation).
- Pour construire les chunks avec chevauchement, pensez à un parcours par fenêtre glissante. Quel est l'incrément entre deux fenêtres successives en fonction de `taille_chunk` et `chevauchement` ?
- L'ID de chaque chunk dans ChromaDB doit être unique et traçable vers son document source. Une convention possible : `"{id_document}__chunk_{numero}"`.
- Lors de la recherche ultérieure, vous récupérerez des chunks — les métadonnées vous permettront de remonter au document source.
- Testez les cas limites : texte plus court que `taille_chunk`, chevauchement nul, chevauchement `taille_chunk` (que doit-il se passer ?).

Tests à écrire (`tests/test_chunking.py`) :

Test	Ce qu'il vérifie
<code>test_decoupe_sans_chevauchement</code>	Un texte de 100 mots découpé en chunks de 30 → 4 chunks
<code>test_decoupe_avec_chevauchement</code>	Les mots de transition apparaissent bien dans deux chunks consécutifs
<code>test_chunk_vide</code>	Un texte vide retourne une liste vide
<code>test_texte_plus_court_que_chunk</code>	Un texte de 50 mots avec <code>taille_chunk=200</code> retourne un seul chunk
<code>test_metadonnees_source</code>	Chaque chunk indexé dans ChromaDB a bien l'ID du document parent dans ses métadonnées

Question de réflexion : comment choisir la taille optimale des chunks ? Quels facteurs influencent ce choix (fenêtre de contexte du LLM, précision de la recherche, taille de la base) ?

Exercice 0.6 — Reranking : affiner la sélection des passages

Concept La recherche sémantique par bi-encoder (sentence-transformers) est rapide mais approximative : elle compare le vecteur de la requête à chaque

vecteur de document de façon **indépendante**, sans prendre en compte les interactions fines entre les mots de la requête et ceux du document.

Le **reranking** ajoute une seconde étape de sélection avec un modèle plus précis mais plus lent :

Etape 1 - Recherche bi-encoder (rapide, sur toute la base)
 Requête : "délai de retour"
 Base : 1000 chunks
 → Top-20 candidats récupérés (distances vectorielles)

Etape 2 - Reranking cross-encoder (précis, sur les 20 candidats)
 Chaque paire (requête, chunk_i) est évaluée conjointement
 → Score de pertinence 0-1 pour chaque candidat
 → Classement final : top-3 retenus pour le prompt

Pourquoi deux étapes ?

Critère	Bi-encoder	Cross-encoder
Vitesse	Très rapide (comparaison de vecteurs pré-calculés)	Lent (inférence à chaque paire requête-document)
Précision	Approximative	Élevée
Passage à l'échelle	Excellente (millions de documents)	Difficile à grande échelle
Usage	Retrieval initial (candidats)	Reranking final (sélection fine)

Le modèle de cross-encoder recommandé : **cross-encoder/ms-marco-MiniLM-L-6-v2**
 — ~70 Mo, rapide sur CPU.

```
Requête + Chunk_1 → Cross-encoder → Score 0.87 ← retenu
Requête + Chunk_2 → Cross-encoder → Score 0.23
Requête + Chunk_3 → Cross-encoder → Score 0.91 ← retenu
...
Requête + Chunk_20 → Cross-encoder → Score 0.05
```

Votre objectif : créer `rag/reranking.py` avec une fonction `_reclasser_passages(requete, candidats, modele_crossencoder, top_k_final)` qui : 1. Prend en entrée la requête et une liste de candidats textuels (résultat du bi-encoder) 2. Calcule le score cross-encoder pour chaque paire (`requête, candidat_i`) 3. Trie les candidats par score décroissant 4. Retourne les `top_k_final` meilleurs candidats avec leurs scores

Puis modifier `rag/pipeline.py` pour intégrer le reranking :

- Le bi-encoder récupère un nombre élargi de candidats (ex. : `top_k_biencoder = 10`)
- Le cross-encoder en sélectionne les meilleurs (ex. : `top_k_final = 3`)

Indices :

- La bibliothèque `sentence-transformers` fournit directement la classe `CrossEncoder`. Elle s'instancie de façon similaire à `SentenceTransformer`.

- La méthode de prédiction du cross-encoder prend en entrée une liste de paires [(requête, doc1), (requête, doc2), ...]. Comment construire cette liste à partir de votre liste de candidats ?
- Contrairement au bi-encoder, les scores du cross-encoder sont des logits bruts (peuvent dépasser 1 ou être négatifs selon le modèle). Faut-il les normaliser avant de les comparer ?
- L'ordre d'entrée des candidats n'a pas d'importance pour le cross-encoder — il reclasse, il ne filtre pas. C'est vous qui choisissez de ne garder que les `top_k_final` meilleurs après le reclassement.
- Pensez à logger le score du meilleur candidat après reranking pour pouvoir l'analyser dans Evidently AI (un score moyen qui chute indique une dégradation de la qualité de la base de connaissance).

Tests à écrire (dans `tests/test_rag.py`) :

Test	Ce qu'il vérifie
<code>test_reranking_retourne_top_k</code>	La fonction retourne bien <code>top_k_final</code> éléments
<code>test_reranking_ordre_coherent</code>	Le candidat le plus pertinent est bien classé premier
<code>test_reranking_integre_pipeline</code>	Le pipeline avec reranking retourne une réponse non vide

Question de réflexion : dans quels cas le reranking apporte-t-il une valeur ajoutée significative par rapport au bi-encoder seul ? Quand peut-on se passer du reranking ?

Partie 1 — Qualité de code automatisée avec pre-commit

Concepts

pre-commit est un gestionnaire de *hooks* Git. Un hook est un script automatiquement exécuté avant chaque `git commit`. Si un hook échoue, le commit est bloqué.

```
git commit
|
Hooks pre-commit
• trailing-whitespace → espaces en fin de ligne ?
• end-of-file-fixer   → fichier se termine par \n ?
• check-yaml          → YAML valide ?
• check-added-large-files → fichier > 500 Ko committé ?
• detect-private-key  → clé API accidentellement ajoutée ?
• ruff                → erreurs de style Python ?
• ruff-format         → code formaté ?
• mypy                → types corrects ?
|
PASS                FAIL
Commit validé      Commit bloqué + erreurs affichées
```

Outil	Rôle
<code>ruff</code>	Linter Python ultrarapide — remplace flake8, isort, pyupgrade
<code>ruff-format</code>	Formateur Python — remplace black
<code>mypy</code>	Vérification statique des annotations de types
<code>detect-private-key</code>	Détecte les clés API accidentellement committées
<code>check-added-large-files</code>	Bloque les poids de modèle (<code>.pt</code> , <code>.bin</code>)

Exercice 1.1 — Installation et configuration

Votre objectif : configurer pre-commit sur votre dépôt.

Indices :

- La commande d'installation du paquet est `pip install pre-commit`.
- La commande d'activation dans le dépôt Git est `pre-commit install`. Que crée-t-elle dans `.git/hooks/` ?
- Le fichier de configuration s'appelle `.pre-commit-config.yaml`. Il contient une clé `repos` qui liste les dépôts de hooks à utiliser.
- Chaque repo a une clé `rev` qui fixe la version du hook. Cherchez les versions stables actuelles de `pre-commit-hooks`, `ruff-pre-commit` et `mirrors-mypy`.
- Pour `ruff`, l'argument `--fix` permet de corriger automatiquement certaines erreurs. Quel est l'avantage et le risque de cette option ?

Exercice 1.2 — Observation du comportement

Votre objectif : comprendre ce qui se passe concrètement quand un hook échoue.

Indices :

- Ajoutez intentionnellement des espaces en fin de ligne dans un fichier Python, puis tentez un commit. Que se passe-t-il ?
- La commande `pre-commit run --all-files` exécute tous les hooks sans passer par un commit. Utile pour déboguer.
- La commande `pre-commit run trailing-whitespace --all-files` exécute un seul hook ciblé.
- `git commit --no-verify` contourne les hooks. Quand est-ce acceptable ? Quand ne faut-il surtout pas l'utiliser ?

Question de réflexion : quel est l'avantage de bloquer les erreurs au niveau du commit plutôt qu'au niveau de la CI ? Dans quel cas les deux niveaux sont-ils tous deux nécessaires ?

Exercice 1.3 — Configuration de ruff et mypy

Votre objectif : configurer ruff et mypy dans `pyproject.toml`.

Indices :

- La section `[tool.ruff]` dans `pyproject.toml` accepte `line-length` et `target-version`. Quelle version Python ciblez-vous ?
 - La section `[tool.ruff.lint]` accepte `select` (règles activées) et `ignore` (règles désactivées). Les codes de règles sont documentés sur `docs.astral.sh/ruff`. Activez au minimum les catégories E, F, I, B.
 - La section `[tool.mypy]` accepte `warn_return_any`, `warn_unused_configs`, `ignore_missing_imports`. Que font chacune de ces options ?
 - mypy sur `transformers` et `torch` peut générer des faux positifs. Cherchez comment ignorer les imports manquants pour ces bibliothèques spécifiquement.
-

Exercice 1.4 — Hook personnalisé

Votre objectif : écrire un hook `local` qui bloque le commit si un fichier `.pt`, `.bin` ou `.safetensors` est présent dans la zone de staging.

Indices :

- Un hook `local` ne pointe pas vers un dépôt externe — il exécute directement une commande shell.
 - La commande Git pour lister les fichiers en zone de staging est `git diff --cached --name-only`.
 - `grep -E` avec une expression régulière permet de détecter les extensions.
 - Un hook doit retourner le code de sortie 1 pour bloquer, 0 pour laisser passer. Comment enchaîner `grep` et les codes de sortie en bash ?
-

Exercice 1.5 — Intégration dans la CI

Votre objectif : faire tourner pre-commit dans GitHub Actions.

Indices :

- L'action officielle `pre-commit/action@v3.0.1` simplifie l'intégration : elle installe pre-commit et exécute `--all-files` automatiquement.
 - Mettez en cache le répertoire `~/.cache/pre-commit` avec `actions/cache`. Quelle clé de cache utiliser pour invalider le cache quand `.pre-commit-config.yaml` change ?
 - Ce job doit-il s'exécuter avant ou après les tests ? Pourquoi ?
-

Partie 2 — Déploiement local avec FastAPI

Concepts

Un modèle de langage en production est exposé sous la forme d'un **service web** accessible via HTTP. FastAPI est un framework Python moderne qui génère automatiquement une documentation Swagger à `/docs`.

Le pipeline RAG que vous avez construit en Partie 0 devient le cœur de ce service.

```
POST /generate
body : { "prompt": "...", "nb_tokens_max": 200 }
├── 1. Validation Pydantic (types, contraintes)
├── 2. Récupération du prompt Langfuse (version de production)
├── 3. Recherche RAG (ChromaDB + reranking)
├── 4. Génération LLM (Qwen2.5-1.5B-Instruct)
└── 5. Réponse JSON + trace Langfuse + logging
```

Exercice 2.1 — Structure de l'API

Votre objectif : créer `api/app.py` avec une application FastAPI exposant deux endpoints : `GET /health` et `POST /generate`.

Indices :

- FastAPI utilise `pydantic.BaseModel` pour définir les schémas de requête et réponse. Chaque champ peut avoir des contraintes (`Field(ge=1, le=500)` pour les bornes numériques, `min_length=1` pour les chaînes).
- Le décorateur `@app.on_event("startup")` permet de charger le modèle une seule fois au démarrage. Pourquoi est-ce important pour la performance ?
- `GET /health` doit retourner le nom du modèle et le statut du service. Utile pour les health checks automatisés.
- `POST /generate` doit appeler le pipeline RAG de la Partie 0 et retourner le texte généré, le nombre de tokens produits, et les IDs des documents sources utilisés.

Attention : le modèle Qwen2.5-1.5B-Instruct prend 10 à 30 secondes à charger selon la machine. Quel est l'impact si vous le rechargez à chaque requête ? Comment le chargement au démarrage (`startup`) résout-il ce problème ?

Exercice 2.2 — Gestion des erreurs

Votre objectif : gérer proprement les cas d'erreur de l'API.

Indices :

- FastAPI lève automatiquement une `422 Unprocessable Entity` si la validation Pydantic échoue. Testez cela avec `curl` en envoyant un prompt vide ou un `nb_tokens_max` hors bornes.
 - Pour les erreurs métier (modèle non chargé, ChromaDB inaccessible), utilisez `HTTPException` avec le bon code HTTP. Quel code HTTP signifie “service temporairement indisponible” ?
 - `@app.exception_handler(RequestValidationError)` permet de personnaliser le format de la réponse d’erreur Pydantic. Quel format JSON uniformisé proposeriez-vous ?
 - Pensez aux logs : en cas d’erreur, quelles informations faut-il logger pour faciliter le débogage en production ?
-

Exercice 2.3 — Logging des requêtes

Votre objectif : écrire dans un fichier JSON Lines une entrée à chaque appel de `/generate`.

Indices :

- Le format JSON Lines (`.jsonl`) stocke un objet JSON par ligne. C’est le format standard pour les logs structurés en ML.
 - Chaque entrée doit contenir au minimum : horodatage, prompt original, réponse générée, longueur du prompt (en mots), longueur de la réponse, durée de traitement en ms, IDs des documents RAG utilisés.
 - `time.perf_counter()` est plus précis que `time.time()` pour mesurer une durée.
 - Le fichier de log doit être ouvert en mode *append* ("`a`") et chaque ligne doit se terminer par `\n`.
 - Que se passe-t-il si deux requêtes arrivent simultanément et écrivent dans le même fichier ? (question avancée : vous pouvez ignorer ce cas pour l’instant)
-

Exercice 2.4 — Conteneurisation Docker (optionnel)

Votre objectif : packager l’API dans un conteneur Docker.

Indices :

- Utilisez une image de base `python:3.11-slim` pour minimiser la taille.
- Les poids du modèle LLM (~270 Mo) et la base ChromaDB ne doivent **pas** être copiés dans l’image — ils doivent être montés via des volumes Docker ou téléchargés au démarrage.
- La variable d’environnement `HF_HOME` contrôle où HuggingFace stocke ses fichiers. À définir dans le Dockerfile.

- Créez un `.dockerignore` pour exclure `.git`, `.venv`, `__pycache__`, et les fichiers de modèle (`.pt`, `.bin`).
- La commande de démarrage doit lancer `uvicorn` en mode production (sans `--reload`), avec `--host 0.0.0.0`.

Partie 3 — Observabilité LLM avec Langfuse

Concepts

Monitoring système vs observabilité LLM Prometheus sait que votre API a mis 6 secondes à répondre. Il ne sait pas *pourquoi*. Langfuse répond à cette question : quelle étape a pris combien de temps, quel prompt exact a été envoyé, quels passages ont été injectés, et quelle était la qualité de la réponse.

```
Niveau 1 - Métriques système (Prometheus)
"La requête a pris 6,2 s. Le CPU était à 78 %."

Niveau 2 - Logs structurés (JSON Lines)
{"prompt": "...", "duree_ms": 6200, "nb_docs": 3}

Niveau 3 - Observabilité LLM (Langfuse)      ← ce que Langfuse apporte
Trace : requête-xyz
├── Span retrieval      : 0,3 s → 3 chunks retrouvés (scores : 0.82, 0.71, 0.55)
├── Span reranking     : 0,4 s → reclassement → [0.91, 0.82, 0.55]
├── Span generation    : 5,5 s → 87 tokens générés
│   ├── prompt_augmenté : "[contexte FAQ-01]... Question : ..."
│   └── réponse         : "Vous pouvez retourner..."
└── Score rouge_1      : 0.41
    Score recall_at_3 : 1
```

Architecture Langfuse

Objet	Rôle	Analogie
Trace	Enregistrement d'une requête complète	Une ligne de log, mais hiérarchique
Span	Sous-opération au sein d'une trace	Un appel de fonction instrumenté
Generation	Span spécialisé pour un appel LLM	Enregistre prompt, réponse, tokens
Score	Métrique attachée à une trace	Note de qualité (manuelle ou automatique)
Prompt	Template versionné dans Langfuse	Gestion centralisée des prompts

Langfuse (open-source et local) Deux options :

- **Cloud** (cloud.langfuse.com) : 50 000 traces/mois offertes, aucune installation

- **Local** (Docker) : auto-hébergé, données sur votre machine, illimité
-

Exercice 3.1 — Lancer Langfuse

Option A — Cloud (recommandée pour démarrer) : 1. Créer un compte sur `cloud.langfuse.com` 2. Créer un projet, récupérer `LANGFUSE_PUBLIC_KEY` et `LANGFUSE_SECRET_KEY` 3. Laisser `LANGFUSE_HOST` vide (pointe vers le cloud par défaut)

Option B — Local avec Docker Compose : Cherchez dans la documentation officielle Langfuse le fichier `docker-compose.yml` de déploiement local. Il nécessite une base PostgreSQL.

Configuration par variables d'environnement :

Créez un fichier `.env` à la racine du projet :

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=https://cloud.langfuse.com
```

Indices :

- Ce fichier **ne doit jamais être commité**. Vérifiez que `.env` est dans votre `.gitignore` avant tout `git add`.
- La bibliothèque `python-dotenv` permet de charger automatiquement `.env` au démarrage du programme (`load_dotenv()`). Installez-la et appelez cette fonction au tout début de `api/app.py`.
- Vérifiez que la connexion fonctionne en instanciant `Langfuse()` et en appelant `langfuse.auth_check()`. Que retourne-t-elle si les clés sont incorrectes ?
- Dans la CI GitHub Actions, les clés seront passées comme **secrets** (Paramètres du dépôt → Secrets → Actions). Jamais en clair dans le YAML.

Règle absolue : une clé API commitée dans un dépôt public est considérée comme compromise, même si elle est supprimée ensuite (l'historique Git la conserve).

Exercice 3.2 — Créer une trace par requête

Votre objectif : enregistrer chaque appel à `POST /generate` comme une trace Langfuse.

Indices :

- `Langfuse()` s'instancie une seule fois au démarrage (comme le modèle LLM). Pourquoi ne pas l'instancier à chaque requête ?

- `langfuse.trace(name=..., input=..., user_id=..., metadata=...)` crée une trace. Le paramètre `name` est ce qui apparaît dans l'interface — choisissez-le descriptif ("**rag-generate**").
- Le `input` de la trace est le prompt brut reçu du client (avant augmentation RAG).
- `metadata` peut contenir tout ce que vous jugez utile pour filtrer les traces dans l'interface : version du modèle, `nb_tokens_max`, horodatage, OS du serveur...
- La trace doit être clôturée avec `trace.update(output=réponse_finale)` avant la fin de la requête.
- `langfuse.flush()` force l'envoi des traces bufferisées. Utile dans les scripts batch (évaluation) pour s'assurer que tout est envoyé avant la fin du processus.

Attention : Langfuse envoie les traces de manière asynchrone (batch). En développement, si le programme se termine trop vite, certaines traces peuvent ne pas être envoyées. Comment éviter cela ?

Exercice 3.3 — Décomposer le pipeline RAG en spans

Votre objectif : enrichir la trace de la requête avec un span par étape du pipeline.

```
Trace : rag-generate
├── Span "retrieval" : début → fin du `rechercher_documents()`
├── Span "reranking" : début → fin du `reclasser_passages()`
└── Generation "llm" : prompt_augmenté → réponse + token usage
```

Indices :

- `trace.span(name="retrieval", input=prompt, ...)` crée un span fils de la trace. Il doit être clôturé avec `span.end(output=...)` après l'opération.
- Pour le span **retrieval** : `input` = prompt de l'utilisateur, `output` = liste des IDs et scores des documents retrouvés, `metadata` = `{"nb_candidats": 10, "seuil": 0.4}`.
- Pour le span **reranking** : `input` = liste des candidats du bi-encoder, `output` = liste reclassée avec scores cross-encoder.
- Pour la génération LLM, utilisez `trace.generation(name="llm", model=..., input=prompt_augmenté, output=réponse)`. La classe `Generation` accepte aussi un paramètre `usage` avec les champs `input_tokens`, `output_tokens`, `total_tokens`. Cela permet à Langfuse d'estimer des coûts (même pour un modèle local).
- Réfléchissez à la granularité : faut-il un span séparé pour le chunking ? Pour l'embedding du prompt ?

Question de réflexion : quels spans sont les plus utiles pour diagnostiquer un problème en production ? Si la latence totale augmente,

comment les spans vous aident-ils à isoler le goulot d'étranglement ?

Exercice 3.4 — Attacher des scores aux traces

Votre objectif : enrichir chaque trace avec des scores de qualité calculables immédiatement (sans annotation humaine).

Scores à attacher :

Nom du score	Valeur	Ce qu'il signale
<code>nb_docs_retrouves</code>	entier (0–5)	Combien de documents pertinents ont été trouvés
<code>score_reranking_max</code>	flottant (0–1)	Score du meilleur document après reranking
<code>longueur_reponse_mots</code>	entier	Longueur de la réponse (proxy de complétude)
<code>hors_domaine</code>	0 ou 1	1 si aucun document n'a dépassé le seuil de pertinence

Indices :

- `trace.score(name="nb_docs_retrouves", value=3)` attache un score à la trace. Langfuse agrège ces scores et les affiche dans un tableau de bord.
 - Le score `hors_domaine` est particulièrement intéressant : il permet de filtrer dans l'interface toutes les requêtes pour lesquelles le RAG n'a pas trouvé de contexte pertinent. Quelle action cela devrait-il déclencher ?
 - En **Partie 7** (évaluation), vous attacherez aussi les scores ROUGE-L et Recall@k aux traces correspondantes, créant un lien entre la trace de production et sa note de qualité.
 - Comment retrouver la trace d'une requête spécifique pour y attacher un score calculé a posteriori ? Langfuse permet de récupérer une trace par son ID ou par des métadonnées.
-

Exercice 3.5 — Gestion des prompts

Votre objectif : stocker le template de prompt système dans Langfuse et le récupérer dynamiquement au démarrage de l'API.

Pourquoi gérer les prompts dans Langfuse ?

Sans gestion centralisée :
Modifier le prompt → modifier le code → commit → CI → redéploiement
Durée : ~10 minutes, risque de régression

Avec Langfuse Prompt Management :
Modifier le prompt → sauvegarder dans l'interface Langfuse
L'API récupère la dernière version au démarrage
Durée : ~30 secondes, aucun redéploiement

Indices :

- Dans l'interface Langfuse (section *Prompts*), créez un prompt nommé "assistant-service-client" avec le template de votre prompt système (incluant les variables `{{contexte}}` et `{{question}}`).
- `langfuse.get_prompt("assistant-service-client")` récupère la version de production du prompt. Cette méthode doit être appelée au démarrage de l'API (`@app.on_event("startup")`).
- La méthode `.compile(contexte=..., question=...)` remplace les variables dans le template. Comparez cette approche à un simple `str.format()` — quels avantages Langfuse apporte-t-il ?
- Langfuse garde un historique de toutes les versions du prompt. Quelle est la valeur de cet historique si la qualité des réponses se dégrade après une modification ?
- Ajoutez un fallback : si Langfuse est inaccessible au démarrage, chargez le prompt depuis un fichier local `prompts/default_prompt.txt`.

Exercice 3.6 — Exporter les traces vers Evidently AI

Votre objectif : remplacer le logging JSON Lines manuel par une exportation depuis Langfuse pour alimenter Evidently AI.

Indices :

- Langfuse expose une API REST pour exporter les traces. La méthode Python `langfuse.fetch_traces(limit=500)` retourne les traces sous forme d'objets Python.
- Parcourez les traces retournées, extrayez les champs utiles pour Evidently (`prompt`, `longueur_prompt`, `nb_docs_retrouves`, `score_reranking_max`) et construisez un `DataFrame` pandas.
- Ce `DataFrame` remplace le `_charger_jsonl(fichier_production)` de la Partie 4 (Evidently). Vous pouvez continuer à utiliser le même fichier de référence `corpus_reference.jsonl`.
- Pensez à gérer la pagination : `langfuse.fetch_traces()` peut retourner des pages si le nombre de traces est élevé. Vérifiez les paramètres `page` et `limit`.
- Le script `monitoring/analyse_derive.py` doit accepter une option en ligne de commande : `--source langfuse` (appel API) ou `--source fichier` (JSON Lines local). Quelle bibliothèque Python standard gère les arguments en ligne de commande ?

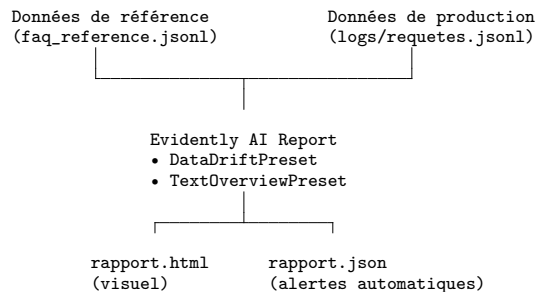
Partie 4 — Dérive des données avec Evidently AI

Concepts

Dérive des données (*data drift*) Lorsqu’un système est en production, les données qu’il reçoit évoluent. Pour un LLM de service client :

Type de dérive	Exemple concret
Dérive de longueur	Les prompts deviennent plus longs (les utilisateurs posent des questions plus détaillées)
Dérive de vocabulaire	Apparition de nouveaux termes (nouveau produit lancé)
Dérive sémantique	Changement de sujet (de “suivi de commande” vers “réclamation qualité”)
Dérive du taux de hits RAG	Le pipeline RAG ne trouve plus de documents pertinents

Evidently AI compare une distribution de *référence* (comportement historique attendu) à une distribution *courante* (production récente) et génère des rapports HTML et JSON.



Exercice 4.1 — Données de référence

Votre objectif : constituer le fichier `data/corpus_reference.jsonl` à partir de requêtes représentatives.

Indices :

- Ce fichier doit contenir au minimum une vingtaine d’entrées couvrant les différentes catégories de la FAQ.

- Chaque entrée doit avoir les mêmes colonnes que les logs de production : `prompt`, `longueur_prompt`, `longueur_reponse`, `nb_docs_rag_trouves`.
 - La colonne `nb_docs_rag_trouves` est une métrique spécifique au RAG : combien de documents ont été retrouvés avec un score au-dessus du seuil. Comment cela peut-il servir à détecter une dérive ?
-

Exercice 4.2 — Rapport de dérive

Votre objectif : créer `monitoring/analyse_derive.py` qui charge les deux datasets et génère un rapport Evidently AI.

Indices :

- Installez `evidently` (`pip install evidently`).
 - `Report` est la classe principale. Elle prend une liste de `metrics` ou de `metric_preset`.
 - `DataDriftPreset()` analyse la dérive sur toutes les colonnes numériques et textuelles.
 - `TextOverviewPreset(column_name="prompt")` fournit des statistiques spécifiques aux colonnes texte (longueur, vocabulaire, etc.).
 - La méthode `report.run(reference_data=..., current_data=..., column_mapping=...)` exécute l'analyse.
 - `ColumnMapping` permet de déclarer quelles colonnes sont numériques, quelles sont textuelles, et quelle est la colonne cible.
 - Sauvegardez les résultats en HTML (pour la visualisation) et en JSON (pour les alertes automatiques).
-

Exercice 4.3 — Simulation de dérive et détection automatique

Votre objectif : simuler une dérive, puis écrire une fonction qui lit le JSON Evidently et déclenche une alerte si un seuil est dépassé.

Indices :

- Pour simuler une dérive, envoyez à l'API des requêtes hors domaine (sujets non couverts par la FAQ) ou des requêtes très longues. Régénérez ensuite le rapport.
- Le JSON Evidently a une structure `{"metrics": [...]}`. Chaque métrique a un champ `metric` (le nom de classe) et un champ `result` (les valeurs calculées).
- Pour `DatasetDriftMetric`, le résultat contient `share_of_drifted_columns` (entre 0 et 1). Quel seuil choisir ?
- Votre fonction doit retourner `True` si une alerte est déclenchée et afficher un message clair. Elle doit aussi retourner `False` sans crasher si le fichier JSON ne contient pas la métrique attendue.

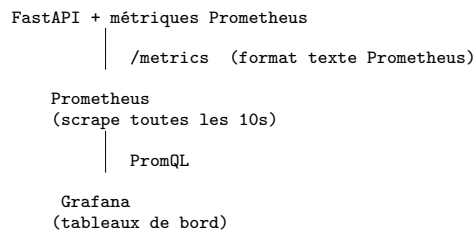
- Ajoutez une colonne `nb_docs_rag_trouves` dans les logs et vérifiez si cette métrique dérive aussi (cela indiquerait que les requêtes s'éloignent du domaine de la base de connaissance).

Question de réflexion : quelle est la différence entre dérive des données (*data drift*) et dérive du modèle (*model drift*) ? Evidently AI peut-il détecter les deux ?

Partie 5 — Surveillance système avec Prometheus et Grafana

Concepts

La dérive des données est une forme de surveillance *métier*. La surveillance *technique* s'assure que le service répond dans les délais et que les ressources système sont dans les limites acceptables.



Métriques clés pour un pipeline RAG :

Métrique	Description
<code>http_request_duration_seconds</code>	Latence totale (réseau + RAG + LLM)
<code>rag_retrieval_duration_seconds</code>	Durée de la recherche ChromaDB uniquement
<code>llm_generation_duration_seconds</code>	Durée de l'inférence LLM uniquement
<code>rag_docs_retrieved_total</code>	Nombre de documents retrouvés par requête
<code>llm_tokens_generated_total</code>	Tokens générés (coût computationnel)

Exercice 5.1 — Exposition des métriques

Votre objectif : instrumenter `api/app.py` pour exposer des métriques Prometheus.

Indices :

- Installez `prometheus-fastapi-instrumentator` et `prometheus-client`.
- `Instrumentator().instrument(app).expose(app)` ajoute automatiquement l'endpoint `/metrics` et instrumente tous les endpoints HTTP. Ajoutez cette ligne après la création de `app`.

- Pour les métriques *personnalisées* (durée RAG, durée LLM, documents retrouvés), utilisez les classes `Counter`, `Histogram` et `Gauge` de `prometheus_client`.
- Un `Counter` ne peut qu'augmenter (ex. : nombre total de tokens). Un `Histogram` mesure des distributions (ex. : durée). Une `Gauge` peut monter et descendre (ex. : nombre de documents dans la collection).
- Observez l'endpoint `/metrics` dans un navigateur après quelques requêtes. Identifiez vos métriques personnalisées dans la sortie.

Exercice 5.2 — Prometheus et Grafana en local

Votre objectif : lancer la pile de surveillance avec Docker Compose.

Indices :

- Créez `monitoring/prometheus.yml` avec une section `scrape_configs` pointant vers votre API FastAPI. Le `job_name` identifie la source dans Prometheus.
- Sur macOS et Windows, `host.docker.internal` permet au conteneur Docker d'atteindre `localhost` de la machine hôte. Sur Linux, utilisez l'adresse IP de l'interface Docker (`ip addr show docker0`).
- Dans `monitoring/docker-compose.yml`, déclarez deux services : `prom/prometheus` et `grafana/grafana`. Montez `prometheus.yml` en volume lecture seule dans Prometheus.
- Le mot de passe admin Grafana peut être passé via la variable d'environnement `GF_SECURITY_ADMIN_PASSWORD`.
- Vérifiez dans l'interface Prometheus (<http://localhost:9090/targets>) que le statut de votre cible est UP.

Exercice 5.3 — Tableau de bord Grafana

Votre objectif : créer un tableau de bord avec au moins 4 panneaux.

Indices :

- Commencez par ajouter Prometheus comme source de données dans Grafana (Configuration → Data sources).
- Les requêtes PromQL sont le langage de requête de Prometheus. `rate(compteur[5m])` calcule le taux par seconde sur 5 minutes. `histogram_quantile(0.95, ...)` calcule le percentile 95.
- Un panneau *Time series* est adapté pour les métriques qui évoluent dans le temps. Un panneau *Stat* affiche une seule valeur.
- Exportez votre dashboard en JSON (Dashboard → Share → Export) et sauvegardez-le dans `monitoring/grafana_dashboard.json` pour le versionner.

Panneaux suggérés : 1. Latence au p95 de l'endpoint `/generate` (histogram_quantile) 2. Débit de requêtes (req/s) 3. Nombre moyen de documents RAG retrouvés par requête 4. Durée de la recherche ChromaDB vs durée de génération LLM (comparaison)

Question de réflexion : quelle est la différence entre *monitoring* et *observabilité* ? En quoi les métriques Prometheus et les logs JSON Lines constituent-ils deux niveaux complémentaires ?

Exercice 5.4 — Alertes Prometheus (optional)

Votre objectif : définir des règles d'alerte dans un fichier `monitoring/alertes.yml`.

Indices :

- Les règles d'alerte Prometheus sont définies sous la clé **groups**, chaque groupe contenant des **rules**.
- Chaque règle a un **alert** (nom), une **expr** (condition PromQL), un **for** (durée avant déclenchement) et des **annotations**.
- Pour référencer ce fichier, ajoutez une clé **rule_files** dans **prometheus.yml**.
- Pensez à des alertes spécifiques au RAG : que faire si **rag_docs_retrieved_total** chute brusquement ? Cela pourrait indiquer que la base ChromaDB est vide ou inaccessible.

Partie 6 — Intégration continue avec GitHub Actions

Concepts

GitHub Actions est un système d'**intégration continue** (CI) intégré à GitHub. À chaque **git push**, il exécute automatiquement une série de vérifications définies dans des fichiers YAML appelés *workflows*.

```
git push
```

GitHub Actions

- Workflow codecheck.yaml
 - Job : pre-commit --all-files
 - ruff, mypy, trailing-whitespace...
- Workflow tests.yaml (démarré après codecheck si needs: est utilisé)
 - Job : tests sur ubuntu-latest / Python 3.10
 - Job : tests sur ubuntu-latest / Python 3.11
 - Job : tests sur ubuntu-latest / Python 3.12
 - Job : tests sur windows-latest / Python 3.10
 - Job : tests sur windows-latest / Python 3.11
 - Job : tests sur windows-latest / Python 3.12
 - (uniquement ubuntu / 3.11)

Rapport de couverture de code

Vocabulaire GitHub Actions :

Terme	Définition
Workflow	Fichier YAML dans <code>.github/workflows/</code> — décrit l'ensemble du pipeline CI
Event	Ce qui déclenche le workflow (<code>push</code> , <code>pull_request</code> , <code>schedule...</code>)
Job	Unité d'exécution indépendante — tourne sur un <i>runner</i> dédié
Step	Commande ou action exécutée dans un job
Runner	Machine virtuelle fournie par GitHub (<code>ubuntu-latest</code> , <code>windows-latest</code> , <code>macos-latest</code>)
Action	Bloc réutilisable (<code>actions/checkout@v4</code> , <code>actions/cache@v4...</code>)
Matrix	Génère automatiquement plusieurs jobs à partir de combinaisons (ex. : <code>OS × Python</code>)
Artifact	Fichier produit par un job et téléchargeable après le run (rapport HTML, résultats)
Secret	Variable d'environnement chiffrée stockée dans les paramètres du dépôt

Exercice 6.1 — Anatomie d'un workflow

Votre objectif : créer le fichier `.github/workflows/codecheck.yaml` qui exécute pre-commit à chaque push.

Structure type d'un workflow :

```

non_workflow      ← nom affiché dans l'onglet Actions de GitHub

on:               ← événements déclencheurs
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  nom_du_job:      ← identifiant interne du job
    name: ...      ← nom affiché dans l'interface
    runs-on: ...   ← type de runner
    steps:
      - uses: ...   ← action externe (depuis le marketplace)
      - name: ...
        run: ...    ← commande shell

```

Indices :

- La première step de presque tout workflow est `actions/checkout@v4`. Pourquoi est-elle indispensable ?
- L'action `actions/setup-python@v5` installe Python sur le runner. Elle accepte un paramètre `python-version`.
- L'action `pre-commit/action@v3.0.1` installe et exécute pre-commit automatiquement. Que lit-elle pour savoir quels hooks lancer ?
- Pour mettre en cache les environnements pre-commit, utilisez `actions/cache@v4` avec `path: ~/.cache/pre-commit`. La clé de cache doit changer quand `.pre-commit-config.yaml` change — quelle expression permet de calculer le hash d'un fichier dans GitHub Actions ?
- Ce job de qualité de code doit-il s'exécuter *avant* les tests ou *en parallèle* ? Justifiez votre choix.

Exercice 6.2 — Matrice OS × Python

Votre objectif : configurer `.github/workflows/tests.yaml` avec une matrice qui teste sur 2 systèmes d'exploitation et 3 versions de Python.

Concept de la matrice :

Plutôt que d'écrire 6 jobs identiques, GitHub Actions génère automatiquement les combinaisons à partir de la clé `strategy.matrix` :

```
matrix:
  os: [ubuntu-latest, windows-latest]
  python-version: ["3.10", "3.11", "3.12"]
```

→ 2 × 3 = 6 jobs générés automatiquement

Chaque variable de la matrice est accessible dans le job via `${{ matrix.os }}` et `${{ matrix.python-version }}`.

Indices :

- Utilisez `runs-on: ${{ matrix.os }}` pour que chaque job utilise le bon runner.
- La clé `fail-fast: false` sous `strategy` empêche d'annuler tous les autres jobs si l'un échoue. Pourquoi est-ce important pour une matrice ?
- Le nom du job peut inclure les variables de matrice pour les distinguer dans l'interface : `name: Tests (${{ matrix.os }} / Python ${{ matrix.python-version }}`).
- Attention : sous Windows, certaines commandes shell changent. `pip install` reste identique, mais les chemins utilisent `\` au lieu de `/`. Comment rendre votre workflow portable ?

Piège courant : les versions Python dans la matrice doivent être des chaînes entre guillemets ("3.10", "3.11"), pas des nombres. Sinon 3.10 est interprété comme 3.1 (suppression du zéro final).

Exercice 6.3 — Mise en cache des dépendances et du modèle

Votre objectif : accélérer la CI en mettant en cache les packages Python et le modèle LLM.

Pourquoi le cache est crucial pour un projet LLM :

```
Sans cache :
  pip install ...           ~2-3 minutes
  HuggingFace download ... ~5-15 minutes (Qwen2.5-1.5B-Instruct, ~3 Go)
  Total : ~5 minutes par job × 6 jobs = 30 minutes de CI
```

```
Avec cache :
  Restauration depuis le cache : ~15 secondes
  Total : < 3 minutes pour les 6 jobs
```

Indices :

- `actions/cache@v4` prend deux paramètres essentiels : `path` (dossier à cacher) et `key` (identifiant unique du cache).
 - Pour le cache des packages pip : le chemin est `~/.cache/pip` et la clé doit changer quand `requirements.txt` ou `requirements_tests.txt` changent.
 - Pour le cache du modèle HuggingFace : le chemin est `~/.cache/huggingface`. La clé doit changer quand le code source change (le nom du modèle est défini dans `src/*.py`). Utilisez `hashFiles('src/**/*.py')` comme partie de la clé.
 - `actions/cache` restaure automatiquement le cache si la clé correspond, et sauvegarde le nouveau cache si la clé est nouvelle (cache miss). Que se passe-t-il lors du *premier* run (aucun cache existant) ?
 - Pour ChromaDB en mode mémoire, il n'y a rien à cacher — la collection est recrée à chaque run de test. Confirmez-le dans vos fixtures pytest.
-

Exercice 6.4 — Séquencement des jobs avec `needs`

Votre objectif : faire dépendre le job de tests du succès du job de qualité de code.

Concept :

Par défaut, tous les jobs d'un workflow s'exécutent EN PARALLÈLE.

```
avec needs:
  job_qualite → (succès) → job_tests
               (échec)  → job_tests annulé
```

Indices :

- La clé `needs: [nom_du_job]` dans la définition d'un job crée une dépendance.
- `needs` accepte aussi une liste de jobs : `needs: [job_a, job_b]`.
- Si le job de qualité échoue (ruff détecte une erreur), les tests ne tournent pas. Pourquoi est-ce une économie de ressources ?

- Il est possible de forcer un job à tourner même si ses dépendances échouent, via `if: always()`. Dans quel cas cela est-il utile ?

Exercice 6.5 — Annulation des runs obsolètes

Votre objectif : configurer la clé `concurrency` pour éviter de gaspiller des minutes CI quand plusieurs commits sont poussés rapidement.

Concept :

Sans `concurrency` :

```

Push 1 → Run A (en cours, ~5 min)
Push 2 → Run B (en cours, ~5 min) ← Run A devient inutile
Push 3 → Run C (en cours, ~5 min) ← Runs A et B deviennent inutiles
→ 3 × 5 min = 15 min consommées

```

Avec `concurrency` + `cancel-in-progress`:

```

Push 1 → Run A (annulé quand Push 2 arrive)
Push 2 → Run B (annulé quand Push 3 arrive)
Push 3 → Run C (complété)
→ ~5 min consommées

```

Indices :

- La clé `concurrency` se place au niveau du workflow (pas dans un job).
- `group` identifie le groupe de concurrence. Utilisez `${{ github.workflow }}`-`${{ github.ref }}` pour grouper par workflow + branche.
- `cancel-in-progress: true` annule le run précédent du même groupe quand un nouveau démarre.
- Faut-il activer `cancel-in-progress` pour la branche `main` ? Réfléchissez aux conséquences si un déploiement était en cours.

Exercice 6.6 — Rapport de couverture de code

Votre objectif : générer un rapport de couverture et le publier comme artefact GitHub Actions.

Concept — qu'est-ce que la couverture de code ?

Couverture = % de lignes de code source exécutées pendant les tests

```

src/tokeniseur.py : 85 % couvert
src/modele.py      : 62 % couvert
rag/recherche.py   : 91 % couvert
rag/chunking.py    : 78 % couvert

```

```

Total              : 79 % couvert

```

Indices :

- `coverage run -m pytest tests/` exécute les tests en instrumentant la couverture.
- `coverage report -m` affiche le rapport texte dans les logs CI. L'option `-m` montre les lignes non couvertes.

- `coverage html` génère un rapport HTML interactif dans un dossier `htmlcov/`.
- `actions/upload-artifact@v4` permet de télécharger ce rapport depuis l'onglet Actions de GitHub après le run.
- `--fail-under=70` fait échouer le step si la couverture est en dessous de 70 %. Ce step ne doit tourner que sur **une** combinaison de la matrice (par ex. ubuntu / Python 3.11) pour éviter 6 rapports redondants. Comment conditionner un step avec `if` : dans GitHub Actions ?
- Faut-il inclure `monitoring/` et `api/` dans la couverture ? Cherchez comment exclure des dossiers dans la configuration `[tool.coverage.run]` de `pyproject.toml`.

Exercice 6.7 — Workflow d'évaluation RAG planifié

Votre objectif : créer `.github/workflows/rag_eval.yaml` qui évalue automatiquement la qualité du RAG selon un calendrier.

Concept :

Certaines vérifications ne doivent pas tourner à chaque push (trop lentes ou trop coûteuses), mais périodiquement — par exemple chaque nuit ou chaque semaine.

Déclencheurs disponibles dans GitHub Actions :

<code>push / pull_request</code>	→ à chaque changement de code
<code>schedule (cron)</code>	→ à une heure fixe
<code>workflow_dispatch</code>	→ manuellement via l'interface GitHub

Indices :

- La syntaxe du déclencheur `schedule` utilise le format cron standard à 5 champs : "0 2 * * 1" signifie "tous les lundis à 2h du matin UTC".
- `workflow_dispatch` permet de déclencher le workflow manuellement depuis l'onglet Actions. Très utile pour les tests ponctuels.
- Ce workflow doit : installer les dépendances, restaurer le cache du modèle, lancer `python evaluation/evaluer_rag.py`, puis publier `evaluation/rapport_eval.json` comme artefact.
- Si les métriques d'évaluation tombent en dessous d'un seuil (`Recall@3 < 0.7`), le step doit retourner un code d'erreur non nul pour faire échouer le workflow. Comment faire cela en Python sans lever d'exception ?

Partie 7 — Évaluation de la qualité du RAG

Concepts

Un pipeline RAG peut sembler fonctionner (il génère du texte) sans pour autant être *correct* (il répond à côté, invente des faits, ignore le contexte fourni). L'évaluation permet de **quantifier** cette qualité.

On distingue deux dimensions :

Evaluation du RAG

- Dimension Retrieval (Récupération)
 - "Les bons documents sont-ils retrouvés ?"
 - Recall@k, Precision@k, MRR, NDCG
- Dimension Génération (Qualité de réponse)
 - "La réponse générée est-elle correcte ?"
 - ROUGE, BLEU (lexicaux)
 - G-Eval / LLM-as-a-judge (sémantiques)

Métriques de retrieval

Métrique	Formule	Interprétation
Recall@k	(docs pertinents retrouvés parmi les k) / (docs pertinents totaux)	Le bon document est-il dans le top-k ?
Precision@k	(docs pertinents parmi les k) / k	Parmi les k résultats, combien sont pertinents ?
MRR (<i>Mean Reciprocal Rank</i>)	Moyenne de $1/(\text{rang du 1er bon résultat})$	À quelle position apparaît le meilleur résultat ?

Exemples concrets pour une requête dont le bon document est en position 2 sur 5 résultats :

- Recall@3 = 1 (le document est dans les 3 premiers)
- Precision@3 = $1/3$ (1 bon document parmi 3)
- MRR = $1/2 = 0.5$ (le bon document est en 2e position)

Métriques de génération

Métrique	Type	Ce qu'elle mesure
ROUGE-L	Lexicale	Chevauchement de la sous-séquence commune la plus longue entre réponse générée et référence
BLEU	Lexicale	Chevauchement de n-grammes entre réponse générée et référence

Métrique	Type	Ce qu'elle mesure
Fidélité	Sémantique	La réponse est-elle ancrée dans les documents fournis (pas d'hallucination) ?
Pertinence	Sémantique	La réponse répond-elle à la question posée ?

Limites des métriques lexicales : ROUGE et BLEU comparent les mots, pas le sens. “Le remboursement prend 5 jours” et “Vous serez remboursé en cinq jours” ont un ROUGE proche de 0 malgré un sens identique. Les métriques sémantiques (LLM-as-a-judge) pallient ce problème — c’est l’objet de l’Exercice 7.6 (Ragas).

Exercice 7.1 — Construire le jeu d’évaluation

Votre objectif : créer `evaluation/jeu_evaluation.jsonl` avec des paires question → réponse de référence.

Format attendu :

```
{
  "id": "eval-01",
  "question": "Sous quel délai puis-je retourner un article ?",
  "reponse_reference": "30 jours",
  "docs_pertinents": ["faq-01"]
},
{
  "id": "eval-02",
  "question": "Comment suivre l'expédition de ma commande ?",
  "reponse_reference": "Un email avec un lien de suivi est envoyé à l'expédition",
  "docs_pertinents": ["faq-02"]
},
...
```

Indices :

- Préparez au minimum 15 paires couvrant les différentes catégories de la FAQ.
- `docs_pertinents` contient la liste des IDs de documents FAQ qui *devraient* être retrouvés par le RAG pour cette question. Cette liste sert à calculer Recall@k et MRR.
- Rédigez les `reponse_reference` de façon concise (1 à 2 phrases) — les métriques lexicales (ROUGE) sont sensibles à la longueur.
- Ajoutez aussi quelques questions *hors domaine* (sans `docs_pertinents`) pour évaluer le comportement du système quand aucune information pertinente n’est disponible.

Exercice 7.2 — Métriques de retrieval (Recall@k et MRR)

Votre objectif : créer `evaluation/evaluer_rag.py` et implémenter les métriques Recall@k et MRR.

Indices :

Recall@k :

- Pour chaque question du jeu d'évaluation, lancez la recherche avec `top_k = k`.
- Vérifiez si au moins un des `docs_pertinents` figure dans les IDs des documents retrouvés.
- Le Recall@k global est la moyenne sur toutes les questions. Comment gérer les questions sans `docs_pertinents` (hors domaine) ?

MRR :

- Pour chaque question, trouvez le rang (position 1, 2, 3...) du premier document pertinent dans les résultats.
- Si aucun document pertinent n'est dans le top-k, la contribution est 0.
- La formule MRR est la moyenne des réciproques de ces rangs.

Structure suggérée pour le rapport de sortie :

```
{
  "recall_at_1": 0.60,
  "recall_at_3": 0.87,
  "recall_at_5": 0.93,
  "mrr": 0.74,
  "nb_questions": 15,
  "timestamp": "2025-01-15T10:30:00Z"
}
```

Indices :

- Itérez sur chaque question, appelez votre fonction de recherche, comparez les IDs retournés avec `docs_pertinents`.
- ChromaDB retourne les documents avec leurs métadonnées — assurez-vous que l'ID original est bien stocké dans les métadonnées lors de l'indexation (Exercice 0.1).
- Pensez à utiliser différentes valeurs de k (1, 3, 5) pour avoir plusieurs points de mesure.

Exercice 7.3 — Métriques de génération (ROUGE)

Votre objectif : calculer le score ROUGE-L entre les réponses générées et les réponses de référence.

Indices :

- Installez la bibliothèque `rouge-score` (`pip install rouge-score`). Elle est légère et ne nécessite pas de modèle supplémentaire.
- La classe `rouge_scorer.RougeScorer` prend en paramètre la liste des variantes à calculer : `"rouge1"`, `"rouge2"`, `"rougeL"`.
- Pour chaque question, générez la réponse du pipeline RAG, puis calculez le score avec la `reponse_reference`.
- Le score ROUGE est un dictionnaire avec `precision`, `recall` et `fmeasure`. La métrique d'intérêt principale est `fmeasure` (F1).

- Que faire pour les questions hors domaine ? La réponse du pipeline devrait être une phrase de refus — comment l'évaluer avec ROUGE ?

Question de réflexion : en quoi le score ROUGE est-il insuffisant pour évaluer un assistant de service client ? Quelle information ne capture-t-il pas ?

Exercice 7.4 — Comparaison RAG vs sans RAG

Votre objectif : évaluer le gain apporté par le RAG en comparant les réponses avec et sans contexte injecté.

Indices :

- Créez une fonction de génération *sans RAG* : elle passe le prompt directement au LLM sans étape de recherche.
- Pour chaque question du jeu d'évaluation, générez deux réponses : avec RAG et sans RAG.
- Calculez ROUGE-L pour les deux et comparez les distributions.
- Pour les questions hors domaine, le RAG devrait produire une réponse de refus alors que le LLM seul inventera probablement une réponse. Comment cela se traduit-il dans les scores ?

Format du rapport de comparaison :

```
{
  "avec_rag":
    {"rouge_l_moyen": 0.42, "recall_at_3": 0.87},
  "sans_rag":
    {"rouge_l_moyen": 0.18},
  "gain_rouge_l": "+0.24",
  "nb_questions": 15
}
```

Question de réflexion : si le ROUGE-L avec RAG est seulement légèrement supérieur à celui sans RAG, est-ce que le RAG est inutile ? Quelles dimensions de qualité ROUGE ne mesure-t-il pas ?

Exercice 7.5 — Seuil d'alerte sur les métriques d'évaluation

Votre objectif : faire échouer le script `evaluer_rag.py` si les métriques tombent en dessous d'un seuil acceptable.

Indices :

- Définissez des seuils minimaux : par exemple Recall@3 0.75 et ROUGE-L 0.30.
- En Python, `sys.exit(1)` retourne un code de sortie non nul et fait échouer le step GitHub Actions qui a lancé le script.
- Affichez clairement quelle métrique a déclenché l'alerte et sa valeur actuelle.

- Ce mécanisme est intégré dans le workflow `rag_eval.yaml` (Exercice 6.7) pour détecter toute régression de qualité.

Exercice 7.6 — Évaluation sémantique avec Ragas (optionnel mais recommandé)

Concept **Ragas** est une bibliothèque open-source spécialisée dans l'évaluation des pipelines RAG. Contrairement à ROUGE qui compare des mots, Ragas calcule des métriques **sémantiques** à l'aide d'un *LLM juge*.

Métrique Ragas	Ce qu'elle mesure	Exemple
Faithfulness	La réponse est-elle ancrée dans les documents fournis ?	Détecte les hallucinations
Answer Relevance	La réponse répond-elle à la question posée ?	Détecte les réponses à côté
Context Precision	Parmi les docs injectés, combien sont vraiment utiles ?	Détecte le bruit dans le contexte
Context Recall	Le contexte couvre-t-il tous les éléments nécessaires ?	Détecte les docs manquants

Exemple concret - requête : "Quel est le délai de retour ?"

Doc injecté : "Les retours sont acceptés sous 30 jours."

Réponse générée : "Vous pouvez retourner vos achats dans un délai d'un mois."

ROUGE-L = 0.05 (peu de mots en commun avec la référence exacte)

Faithfulness = 1.0 (la réponse est bien ancrée dans le doc)

Answer Relevance = 0.95 (répond bien à la question)

Contrainte importante : le LLM juge Ragas nécessite un LLM pour évaluer les métriques sémantiques. Deux approches compatibles avec la contrainte d'usage local :

Approche	LLM juge	Local ?	Qualité
Cloud public	API OpenAI ou Groq		Excellente
Local	Qwen2.5-1.5B-Instruct (le même modèle)		Correcte
Local fort	Qwen2.5-7B-Instruct (si GPU disponible)		Bonne

Limite : utiliser le même modèle comme générateur *et* comme juge crée un biais (le modèle tend à s'évaluer favorablement).

Votre objectif : créer `evaluation/evaluer_rag_ragas.py` qui calcule les métriques Ragas sur votre jeu d'évaluation et compare les résultats avec les scores ROUGE de l'Exercice 7.3.

Indices :

- Installez Ragas : `pip install ragas`. Vérifiez la version dans la documentation (l'API a évolué entre les versions 0.1 et 0.2).
- Ragas attend un dataset avec quatre colonnes obligatoires : `question`, `answer` (la réponse générée), `contexts` (liste des passages injectés), `ground_truth` (la `reponse_reference` de votre jeu d'évaluation).
- Construisez ce dataset en parcourant votre `jeu_evaluation.jsonl` : pour chaque question, exécutez le pipeline RAG, récupérez les chunks injectés (pas seulement la réponse finale) et la réponse générée.
- Pour configurer Ragas avec un LLM local, cherchez dans la documentation comment définir un `LangchainLLM` ou un `LangchainEmbeddings` pointant vers votre modèle HuggingFace.
- Comparez dans le rapport final : Faithfulness élevé + ROUGE faible = le modèle paraphrase correctement. Faithfulness faible = hallucinations.
- Attachez les scores Ragas aux traces Langfuse correspondantes (comme pour ROUGE en Exercice 7.5).

Tests minimaux (`tests/test_eval.py`) :

Test	Ce qu'il vérifie
<code>test_dataset_ragas_structure</code>	Le dataset construit a bien les 4 colonnes requises
<code>test_ragas_scores_bornes</code>	Les scores Faithfulness et Answer Relevance sont entre 0 et 1

Question de réflexion : dans quels cas Faithfulness et ROUGE-L divergent-ils fortement ? Que dit cette divergence sur la qualité du modèle ?

Livrables attendus

Livable	Description	Critères d'évaluation
L1 — Dépôt Git	Dépôt GitHub public	Structure correcte, commits atomiques, README clair
L2 — RAG de base	Pipeline RAG fonctionnel (Ex. 0.1 à 0.4)	Requête pertinente → document correct retrouvé

Livrable	Description	Critères d'évaluation
L3 — Chunking	Découpage et indexation de documents longs (Ex. 0.5)	Tests unitaires passants, chevauchement correct
L4 — Reranking	Pipeline bi-encoder + cross-encoder (Ex. 0.6)	Ordre amélioré par rapport au bi-encoder seul
L5 — pre-commit	Hooks configurés et actifs	Tous les hooks passent sur main
L6 — CI GitHub Actions	3 workflows : tests, codecheck, rag_eval	Badge CI vert, matrice OS×Python, cache actif
L7 — API	Endpoints /generate (RAG + Langfuse) et /health	Swagger accessible, traces visibles dans Langfuse
L8 — Langfuse	Traces RAG avec spans + scores + prompt versionné	Dashboard Langfuse peuplé, export vers Evidently
L9 — Évaluation RAG	Métriques Recall@k, MRR, ROUGE-L (Ex. 7.1 à 7.5)	Rapport JSON + scores attachés aux traces Langfuse
L10 — Dérive	Script Evidently AI + rapport HTML	Dérive simulée détectée, alerte déclenchée
L11 — Monitoring	Tableau de bord Grafana exporté en JSON	4 panneaux minimum (RAG + LLM + système)
L12 — Rapport	Document écrit (12 pages)	Voir grille ci-dessous

Grille d'évaluation du rapport

Critère	Barème
Qualité et clarté de la rédaction	/3
Justification des choix techniques (embedding, chunk, seuils, prompts)	/3
Analyse des traces Langfuse : diagnostics, scores, insights observés	/3
Analyse et interprétation des métriques d'évaluation RAG	/3
Analyse critique des limites (SmolLM2, chunking, reranking, hallucinations)	/3

Critère	Barème
Réponses aux questions de réflexion intégrées aux exercices	/3
Originalité / approfondissement	/2
Total	/20

Questions de réflexion — récapitulatif

Les réponses à ces questions doivent figurer dans le rapport écrit.

1. **Embedding de la question vs de la réponse** : dans la FAQ, pourquoi encode-t-on la *question* et non la *réponse* pour l'indexation ?
2. **Taille de chunk et chevauchement** : comment choisir ces hyperparamètres ? Quels critères guident ce choix ? Comment les évaluer expérimentalement ?
3. **Bi-encoder vs cross-encoder** : pourquoi ne pas utiliser directement un cross-encoder sur toute la base ? Quand le reranking est-il superflu ?
4. **Monitoring système vs observabilité LLM** : Prometheus vous dit que la requête a pris 6 secondes. Langfuse vous dit quelle étape a pris combien de temps. Quelles décisions opérationnelles chacun permet-il de prendre que l'autre ne permet pas ?
5. **Gestion des prompts** : quels risques introduit le fait de stocker les prompts directement dans le code source ? Comment Langfuse Prompt Management les atténue-t-il ?
6. **Score hors_domaine en production** : si ce score augmente progressivement sur plusieurs jours, quelle hypothèse formulez-vous sur l'évolution des requêtes des utilisateurs ? Quelle action corrective proposeriez-vous ?
7. **Limites de ROUGE pour l'évaluation d'un LLM** : quelles dimensions de qualité ROUGE ne capte-t-il pas ? Quelles alternatives existent ?
8. **pre-commit vs CI** : à quoi sert de lancer pre-commit localement si la CI fait les mêmes vérifications ?
9. **Matrice OS × Python en CI** : quelle est la valeur ajoutée de tester sur Windows *et* Linux ? Donnez un exemple concret de bug détecté uniquement sur un OS.
10. **Dérive des données vs dérive du modèle** : expliquer la différence. Comment le score `hors_domaine` Langfuse et la dérive Evidently sont-ils complémentaires pour détecter ces deux phénomènes ?

11. **Limites du modèle Qwen2.5-1.5B-Instruct** : en analysant vos traces Langfuse, quels patterns d'erreurs avez-vous observés (répétitions, ignorance du contexte RAG, hallucinations) ? Comment un modèle plus grand changerait-il les résultats ? Qu'est-ce que les scores Ragas (Faithfulness, Answer Relevance) révèlent que ROUGE ne montrait pas ?

Ressources

Documentation officielle

Ressource	URL
ChromaDB	https://docs.trychroma.com
sentence-transformers	https://www.sbert.net
cross-encoder/ms-marco-MiniLM-L-6-v2	https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2
rouge-score	https://pypi.org/project/rouge-score/
FastAPI	https://fastapi.tiangolo.com
Evidently AI	https://docs.evidentlyai.com
Prometheus	https://prometheus.io/docs/introduction/overview/
Grafana	https://grafana.com/docs/grafana/latest/
pre-commit	https://pre-commit.com
Ruff	https://docs.astral.sh/ruff/
GitHub Actions — syntaxe workflow	https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions
GitHub Actions — mise en cache	https://docs.github.com/en/actions/writing-workflows/choosing-what-your-workflow-does/caching-dependencies-to-speed-up-workflows
Langfuse — documentation	https://langfuse.com/docs
Langfuse — Python SDK	https://langfuse.com/docs/sdk/python
Langfuse — Prompt Management	https://langfuse.com/docs/prompts/get-started
Langfuse — self-hosting	https://langfuse.com/docs/deployment/local

Modèles recommandés (sans clé API)

LLM de génération Le choix du modèle a un impact direct sur la qualité observable du RAG. Un modèle trop petit ignorera le contexte injecté et produira des réponses répétitives.

Modèle	Taille	RAM min.	Qualité RAG
distilgpt2	~85 Mo	2 Go	Très faible
HuggingFaceTB/SmolLM2-135M	~270 Mo	2 Go	Faible
Qwen/Qwen2.5-1.5B-Instruct	~3 Go	8 Go	Correcte
Qwen/Qwen2.5-3B-Instruct	~6 Go	12 Go	Bonne
mistralai/Mistral-7B-Instruct-v0.3	~14 Go	GPU recommandé	Excellente

Pourquoi Qwen2.5-1.5B-Instruct ? C'est le modèle le plus léger qui soit *instruction-tuned* : il suit les instructions du prompt système, utilise correctement le contexte RAG injecté et produit des réponses cohérentes. Les modèles plus petits (SmolLM2-135M, distilgpt2) ignorent souvent le contexte et répètent le prompt, ce qui fausserait les conclusions sur la qualité du RAG.

Modèles d'embedding et de reranking

Modèle	Taille	Rôle
all-MiniLM-L6-v2	~80 Mo	Embedding bi-encoder (recherche)
paraphrase-multilingual-MiniLM-L12-v2	~120 Mo	Embedding multilingue
cross-encoder-ms-marco-MiniLM-L-6-v2	~70 Mo	Cross-encoder pour le reranking

Dépendances Python

```
# requirements.txt
transformers>=4.40
torch>=2.2
sentence-transformers>=3.0
chromadb>=0.5
fastapi>=0.111
uvicorn[standard]>=0.29
pydantic>=2.7
prometheus-fastapi-instrumentator>=6.1
prometheus-client>=0.20
evidently>=0.4
rouge-score>=0.1.2
langfuse>=2.0
python-dotenv>=1.0
pandas>=2.0

# requirements_tests.txt
pytest>=8.0
coverage>=7.5
pre-commit>=3.7

# requirements_eval.txt (optionnel - exercice 7.6)
ragas>=0.2
```